Knowing When to Act: A Hybrid Classification–Generation System for Proactive Audio Engagement; Checkpoint I

Bailey Malota
Computer Science
Virginia Tech
Springfield, Virginia
baileym04@vt.edu

Jaemyung Yu
Computer Science
Virginia Tech
Arlington, Virginia
jaemyungy25@vt.edu

Martin Balla
Computer Science
Virginia Tech
Arlington, Virginia
martinb04@vt.edu

Maximilian Ilyev
Computer Science
Virginia Tech
Sterling, Virginia
maximilianilyev@vt.edu

Abstract
Audio-aware assistants in homes, workplaces, and care settings must decide not only what they hear but whether responding would help. We are building a system for Track II of the HEARTBEAT Challenge [1], in which a system listens to an unfolding stream of environmental or human non-speech audio and must decide whether and when to engage and what to say. Our approach decouples deciding from speaking: a frozen pretrained audio encoder feeds a lightweight causal temporal model that predicts actions frame by frame and derives the engagement time, while a template, retrieval, or compact LoRA-tuned language model generates the response conditioned on the predicted action and detected sound events. Because the challenge data had not been released at this checkpoint, we built the full pipeline end to end and evaluated it on a public-audio pseudo-challenge set and on a held-out set of synthetic scenes. Causal encoding, a learned task-vocabulary event probe, and threshold calibration by grouped cross-validation each gave measured gains in distribution. However, on unseen sound sources the system first scored below an always-silent baseline. This report describes the system as built, what changed from the proposal and why, the effect of the organizers' October redefinition of Track II, and our plan to the final submission.

I. Introduction

Smart speakers, home-monitoring devices, and assistive technologies increasingly listen continuously to their surroundings. Their value depends on noticing critical moments like a smoke alarm, a kettle left boiling, a fall, a person coughing or crying while also being able to stay quiet the rest of the time. Both failure modes are costly. A missed alarm can be dangerous, while an assistant that interrupts over every door slam or a sound that has already stopped quickly becomes an annoyance that users ignore or switch off. Modern audio models are good at recognizing what a sound is, but recognition alone does not tell a system whether, when, or how to act.

The HEARTBEAT Challenge [1] formalizes this gap. When we wrote our proposal, Track II ("Proactive Audio Engagement") gave a system a 10–90 s segment of non-speech audio. The system had to (i) choose an engagement time, (ii) predict one of four actions (silence, monitor, ask, or intervene), and (iii) when acting, produce a concrete response and a brief rationale. Hard negatives included weak evidence, premature moments, already-resolved events, and scenes where no action is warranted. On October 9 the organizers redefined the track as "Streaming Proactive Understanding." Systems now receive one instruction and then process audio-only or audio-visual input online, chunk by chunk. They may use only what has been observed so far, may respond several times per stream, and must stay explicitly silent otherwise. Scoring is precision, recall, and a joint F1 over responses matched to ground-truth triggers within a tolerance window. Section VI discusses what this changes for us. The core problem is the same: deciding causally when the evidence warrants a response.

This problem is well suited to deep learning. It requires learned acoustic representations robust to varied devices and environments, temporal modeling of how a scene unfolds, and natural-language generation grounded in audio evidence. Our goals for the semester are staged:

1) Build an end-to-end pipeline and internal evaluation, then reproduce the organizer baseline once it is released (Checkpoint 1).
2) Deliver a complete end-to-end hybrid system with template-based responses (Checkpoint 2).
3) Replace templates with a fine-tuned language-model generator and quantify the gain.
4) Submit a valid final container to the challenge and analyze performance under distribution shift.

The challenge data, rules, and scorers were scheduled for October 14 and have moved to October 20. At this checkpoint, goal 1 is therefore met on proxy data only, and baseline reproduction moves to the week after the release. Section V reports what was built and measured in the meantime.

II. Related Work

A. Audio Representation and Event Detection

Large-scale weakly labeled data such as AudioSet [2] enabled general-purpose audio encoders, including CNN-based PANNs [3], the Audio Spectrogram Transformer (AST) [4], and self-supervised BEATs [5]. CLAP [6] aligns audio with text in a shared embedding space. Sound event detection, advanced largely through the DCASE challenges [7], localizes events in time. These models answer what happened and

when, which is necessary but not sufficient for our task. We use them as frozen feature extractors.

B. Audio-Language Models

Audio captioning datasets such as AudioCaps [8] and Clotho [9] link audio to natural language. More recent audio-language models connect an audio encoder to a large language model (LLM): Pengi [10], LTU [11] (built on LLaMA), SALMONN [12] (built on Vicuna), and Qwen2-Audio [13] (built on Qwen). These models can describe and reason about audio, but they are trained to answer when asked and not to decide whether an unprompted response is warranted. They are also large for a no-network, container-based evaluation.

C. Turn-Taking and Proactive Agents

Deciding when to speak has been studied mainly for conversation, from turn-taking models [14] to self-supervised voice activity projection [15]. Proactive dialogue systems [16] and proactive LLM agents [17] study when an assistant should take initiative, but they operate on text or on user activity, not on environmental audio.

D. Our Contribution

Prior work either detects and describes sounds or decides when to speak in a conversation. Track II combines both problems in non-speech scenes. Our contribution is (i) a hybrid architecture that separates the engagement decision from response generation and conditions generation on that decision to keep actions and responses consistent, (ii) a controlled comparison between template retrieval and LLM generation on the same decision backbone, and (iii) an analysis of false activations and robustness to unseen sound sources and acoustic conditions.

III. Approach as Built

Our system has three stages (Fig. 1). The first two produce the timing and action outputs; the third produces the response and rationale. This section describes the system as implemented, and notes where it departs from the proposal.

A. Stage 1: Frozen Audio Encoder

The clip is read on a frame grid with a 0.5 s hop. Each frame is embedded by a frozen AudioSet encoder, which gives a 768-d (BEATs) or 2048-d (PANNs) embedding plus 527 AudioSet class probabilities, cached once in fp16. We re-implemented PANNs CNN14 and BEATs for inference only and checked them against the reference code (to 3e-7 and 1e-6 on clip probabilities).

Change from the proposal: frames are encoded causally. Encoding the whole clip, or 10 s chunks, lets self-attention carry audio from later in a chunk into earlier frames, so the causal Stage 2 sees evidence before it occurs. Each frame is instead encoded from the 4 s of audio ending at its decision time. This was the largest single gain we measured (Table I), and BEATs encoded this way is our provisional Stage 1.

Change from the proposal: events come from a learned probe. Raw AudioSet probabilities made a poor event list: on our pseudo-dev set, 26 of 60 clips had no event above 0.5, and stationary backgrounds (vehicle, rain, speech) outranked the cues that matter. A per-event logistic probe on the frozen features, trained on the event spans in the training labels, now replaces them with task-vocabulary probabilities. Training clips receive out-of-fold probe output, so Stage 2 never trains on cleaner features than it sees at test time. For events with no training spans, an optional hand-written AudioSet-to-task map supplies a calibrated fallback column.

B. Stage 2: Causal Decision Model

A unidirectional GRU reads the per-frame features and outputs a distribution over the four actions at every frame. Per-frame targets are derived as proposed: silence before the engagement time, the clip's action from it onward, and a 1 s band before it excluded from the loss. A test fails the build if future audio can change earlier outputs.

Change from the proposal: the decoder. Our proposed rule, the first frame where any non-silence action crosses a threshold, lost whole clips to one noisy frame. The rule now thresholds $P(\text{act}) = 1 - P(\text{silence})$ for *when*, requires three consecutive frames above the threshold, and picks *which* action by averaging probabilities over the frames after the trigger. That averaging currently looks up to 3 s past the reported time; Section VI explains why this must change. An optional confirmation window waits a few seconds after a trigger and stands down if footsteps or a door are heard. It targets the resolved-event hard negatives.

Change from the proposal: calibration. We proposed tuning the threshold on development data. It is now chosen by cross-validation on the training split, holding out whole groups of clips that share source recordings, so the development set is never used to tune it (Section V).

C. Stage 3: Response and Rationale Generation

A rule-based step thresholds and merges the probe's per-frame event probabilities into timed spans. It builds a structured summary from these spans and Stage 2′s action and time. Because the action is given rather than chosen by the generator, the response cannot contradict it. Generators share one interface:

- *Template:* a fixed sentence per action. It is the floor.
- *Retrieval:* copies the best-matching training reference for the same event and action. This is the bar any LLM must beat.
- *LLM:* Qwen3.5 (0.8B and 2B), zero-shot, few-shot with retrieved examples, or LoRA-tuned [18]. It loads only from local files and falls back to the template if it breaks format; fallbacks are counted, not hidden. The LoRA training data deliberately corrupts summaries (jittered times, dropped and spurious events), so the generator learns to cope with imperfect upstream predictions.

We switched from the proposal's candidates (Qwen2.5 [19], Llama 3.2 [20], Phi-3 [21]) to Qwen3.5 because of its newer small checkpoints. The final choice is still bounded by license terms and measured container size, and is due by the November 20 freeze.

D. Class Imbalance

Most frames warrant no action. Decision training uses the masked, step-based targets above; the threshold calibration trades false activations against misses. For Stage 3 we still treat monitor as a non-response action, as proposed. This refers to the predicted behavior toward the audio event, rather than the absence of program output: the system can output the monitor label and generate text explaining its decision to keep observing without intervening. Our pseudo-challenge set currently attaches text to monitor clips, and we will align it once the released labels settle the question.

IV. System Design

A. Architecture

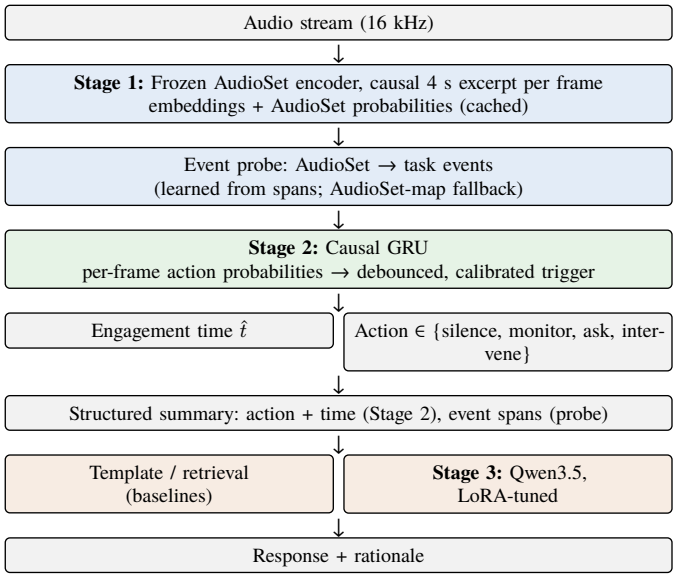


Fig. 1. Pipeline as built. Stages 1–2 produce timing and action; Stage 3 generates the response conditioned on them. The event probe is new since the proposal.

B. Data Sets

Challenge data. The organizers' original description listed 300 h of training, 15 h of development, and 15 h of input-only leaderboard audio, with a hidden test set split 70%/30% into in-distribution and shifted portions. The redefined track page no longer states sizes or the split. Data, rules, scorers, and a baseline are now due October 20.

Pseudo-challenge set. To test the pipeline on real acoustics before the release, we built a labeled proxy set from ESC-50 [22]. ESC-50 has no smoke alarms, doorbells, appliance beeps, kettles, running taps, or falls. We therefore generate those seven event types procedurally (tones, filtered noise, impacts, and voiced groans), each convolved with a random room response. These join five ESC-50 event types (alarm, glass break, door knock, crying baby, coughing) and twelve ESC-50 distractor classes. Scenes of 10–90 s are mixed over ESC-50 backgrounds and labeled by explicit rules. Hard negatives come in four kinds:

- *weak:* the event is mixed below the background;
- *premature:* the evidence becomes sufficient only after the clip ends;
- *resolved:* the sound stops early;
- *resolved by someone:* footsteps or a door land before the system should act.

References are hand-written, so the text metric does not grade a generator against its own wording. Splits use disjoint ESC-50 folds (train 300 clips, 4.1 h; dev 60; leaderboard 30). Clips within a training fold share recordings, which forces grouped cross-validation.

Synthetic test scenes. A separate set of 30 synthetic Track II-style scenes (12 positives, 18 hard negatives) is never used for training. We use it to measure transfer to sound sources and mixing conditions the training data does not contain.

C. Baselines and Evaluation

We compare against (i) the organizer baseline once released, (ii) an always-silent system, and (iii) our template system. Every score our harness prints includes the always-silent score on the same clips and our margin over it, with a paired bootstrap 95% interval. The scorer implements the originally published 0.30 timing + 0.30 action + 0.40 text total. How silence clips count on timing and text, the timing tolerance, and the text metric are configuration switches, not assumptions. Text is scored with token F1 as a download-free stand-in for BERTScore [23]. All model selection uses cross-validation on the training split with whole source groups held out. Results are means and standard deviations over 10 GRU seeds, because the seed spread (±0.03–0.05) is as large as most effects we measure.

D. Languages, Tools, and Compute

The system is implemented in Python with PyTorch, Hugging Face Transformers, and PEFT, in a shared dev container with locked dependencies, with code on GitHub. Experiments so far run on CPU. BEATs causal encoding costs about 0.13 s of CPU per audio second. Full-scale encoding and LoRA tuning will run through Slurm on Virginia Tech ARC (Falcon A30 for encoding and decision training, TinkerCliffs A100 for the LLM), with one shared copy of the data and caches in group storage. Final systems must be no-network OCI containers [1], so model size and latency stay modest.

V. Progress and Findings

All numbers in this section come from proxy data and a stand-in text metric. They measure design choices, not challenge performance.

TABLE I
Stage 1 on pseudo-dev (probe + retrieval, 10 seeds). Total at the CV threshold; trigger event named correctly; event macro-AP.

Encoder	Total	Timing err.	Trigger	AP
PANNs (causal)	0.435 ± 0.044	−1.6 s	23/30	0.418
BEATs (chunks)	0.353 ± 0.039	−3.5 s	24/30	0.436
BEATs (causal)	**0.517 ± 0.027**	**−0.3 s**	**26/30**	**0.580**

Causal encoding matters most. Chunked BEATs fired a median 3.5 s early because attention leaked later audio into

earlier frames. Causal BEATs is better on every component and leads PANNs by about twice the seed spread (Table I).

The event probe removes empty summaries. Clips with no detected event fell from 17 to 0. The triggering event was named correctly in 25 of 30 acting clips, against 0 of 30 possible with AudioSet names.

Grouped calibration closes most of the threshold gap. For PANNs, a fixed threshold of 0.5 fired on about half of silence clips and scored 0.388. Calibrating by randomly assigned folds scored 0.403, because those folds leak shared recordings and choose thresholds that are too low. Holding out whole source groups scored 0.435, against 0.468 for the optimistic threshold tuned on dev itself.

Retrieval is a strong text baseline. On gold summaries (30 acting clips), token F1 was 0.234 for the template, 0.329 for retrieval, 0.183 for Qwen3.5-2B zero-shot, and 0.297 for 2B few-shot. A first LoRA run on the 0.8B model (12 steps on CPU) reached 0.342. It also fixed the zero-shot model's action contradictions and prompt echoing. On held-out sound types, neither prompting variant beat the template.

TABLE II
TRANSFER TO 30 UNSEEN SYNTHETIC SCENES (BEATS CAUSAL, CV THRESHOLD, 10 SEEDS). ALWAYS-SILENT SCORES 0.160.

Training set	Synthetic	Own dev
Pseudo, 5 event types	0.103 ± 0.015	0.516 ± 0.028
Pseudo, 5 types, with fallback	0.117 ± 0.017	0.511 ± 0.028
Pseudo, 12 event types	0.210 ± 0.034	0.386 ± 0.035

Our largest gap is generalization. Trained on the original five ESC-50 event types, the system scored 0.516 in distribution but 0.103 on the synthetic scenes, below the always-silent baseline (Table II). It fired on harmless impacts and missed alarms, doorbells, and kettles, and the probe labeled most short sounds "door knock." Adding the seven generated household event types raised transfer to 0.210. With seed 0, all four intervene scenes were detected, though the margin over always-silent is small. The 12-type development set is harder, so its score is not comparable to the 5-type row. The synthetic scenes also informed which event types we added, so this improvement is optimistic.

Two remedies did not help yet. The AudioSet fallback adds +0.014, about one seed standard deviation; it should matter when labels lack event spans, not here. The confirmation window was rejected by cross-validation in 39 of 40 runs. Forced waits did reduce false activations (0.28 → 0.19 at 4 s on dev) but destroyed timing, because the GRU already fires near the labeled time and any wait adds to it. The remaining failures are ask-level cues (doorbells, beeps, running water) and resolved events.

VI. Impact of the Track Redefinition

The causal design carries over directly: the new rules allow only current and past chunks, which our encoder and GRU already enforce. Four things change:

1) *Output protocol.* We must emit a record for every chunk and timestamp each response at the end of the chunk that produced it, with no backdating. Our 0.5 s frame grid maps onto this naturally. However, the action vote's look-ahead (Section III) would violate it, and must be replaced by a strictly causal choice.
2) *Multiple responses per stream.* The decoder returns one decision per clip and must instead re-arm after each response, without responding twice to the same event.
3) *Metric.* A precision/recall/joint-F1 metric rewards always-silent with zero recall, so the abstention trade-off changes. Our scorer's metric switches let us adopt the official definition without changing the pipeline.
4) *Instructions and modality.* Each stream comes with an instruction, which Stage 3 should condition on. We will target the audio-only subtrack.

Until the rules are published, we treat our four-action formulation as provisional. It may survive as an internal representation even if the output format changes.

VII. Plan to Final Submission

- *Oct 20–27:* read the released rules and scorer; adapt the loader and metric; encode the data on ARC; reproduce the organizer baseline; build the dummy no-network container for size and latency.
- *Oct 28–Nov 10 (Checkpoint 2):* streaming output and a multi-response decoder; train on the real data; fix the causal action choice; template and retrieval end-to-end system; first leaderboard submissions (leaderboard opens Nov 4).
- *Nov 11–20:* LoRA on ARC against retrieval; build a shifted copy of the development set (gain, band-limiting, noise, reverberation); declare models by the Nov 20 freeze.
- *Nov 21–Dec 11:* ablations; freeze the container Dec 3; validate it on the leaderboard before it closes Dec 4; submit the same artifact Dec 11.

Open technical questions are whether to train the GRU to fire at cue onset so that a confirmation window can pay off, and whether a hazard (time-to-engagement) formulation handles silence clips better than per-frame classification.

References

[1] HEARTBEAT Challenge Organizers, "HEARTBEAT Challenge 2027: Human-Centered Audio Agents." [Online]. Available: https://heartbeat-challenge.github.io/

[2] J. F. Gemmeke *et al.*, "Audio Set: An ontology and human-labeled dataset for audio events," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 776–780.

[3] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley, "PANNs: Large-scale pretrained audio neural networks for audio pattern recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2880–2894, 2020.

[4] Y. Gong, Y.-A. Chung, and J. Glass, "AST: Audio Spectrogram Transformer," in *Proc. Interspeech*, 2021, pp. 571–575.

[5] S. Chen *et al.*, "BEATs: Audio pre-training with acoustic tokenizers," in *Proc. Int. Conf. Machine Learning (ICML)*, 2023.

[6] B. Elizalde, S. Deshmukh, M. Al Ismail, and H. Wang, "CLAP: Learning audio concepts from natural language supervision," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2023.

[7] A. Mesaros, T. Heittola, T. Virtanen, and M. D. Plumbley, "Sound event detection: A tutorial," *IEEE Signal Processing Magazine*, vol. 38, no. 5, pp. 67–83, 2021.

[8] C. D. Kim, B. Kim, H. Lee, and G. Kim, "AudioCaps: Generating captions for audios in the wild," in *Proc. NAACL-HLT*, 2019.
[9] K. Drossos, S. Lipping, and T. Virtanen, "Clotho: An audio captioning dataset," in *Proc. IEEE Int. Conf. Acoustics, Speech and Signal Processing (ICASSP)*, 2020.
[10] S. Deshmukh, B. Elizalde, R. Singh, and H. Wang, "Pengi: An audio language model for audio tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
[11] Y. Gong, H. Luo, A. H. Liu, L. Karlinsky, and J. Glass, "Listen, think, and understand," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.
[12] C. Tang *et al.*, "SALMONN: Towards generic hearing abilities for large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.
[13] Y. Chu, J. Xu, Q. Yang, and others, "Qwen2-Audio technical report," *arXiv preprint arXiv:2407.10759*, 2024.
[14] G. Skantze, "Turn-taking in conversational systems and human-robot interaction: A review," *Computer Speech and Language*, vol. 67, p. 101178, 2021.
[15] E. Ekstedt and G. Skantze, "Voice activity projection: Self-supervised learning of turn-taking events," in *Proc. Interspeech*, 2022.
[16] Y. Deng, W. Lei, W. Lam, and T.-S. Chua, "A survey on proactive dialogue systems: Problems, methods, and prospects," in *Proc. Int. Joint Conf. Artificial Intelligence (IJCAI)*, 2023.
[17] Y. Lu and others, "Proactive agent: Shifting LLM agents from reactive responses to active assistance," *arXiv preprint arXiv:2410.12361*, 2024.
[18] E. J. Hu *et al.*, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.
[19] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.
[20] A. Grattafiori and others, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
[21] M. Abdin and others, "Phi-3 technical report: A highly capable language model locally on your phone," *arXiv preprint arXiv:2404.14219*, 2024.
[22] K. J. Piczak, "ESC: Dataset for Environmental Sound Classification," in *Proc. 23rd ACM Int. Conf. Multimedia*, 2015, pp. 1015–1018.
[23] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020.

Appendix

Task Assignment

Member	Responsibilities
Maximilian Ilyev (Lead)	**Primary, evaluation and delivery:** scoring harness, end-to-end parameter sweeps, offline container, challenge submissions, coordination, report editing. **Secondary, Stage 2:** threshold and decoder tuning against the end-to-end score.
Jaemyung Yu	**Primary, Stage 1:** data transfer to ARC, preprocessing, encoder selection, chunking of long clips, embedding and event-probability caching. **Secondary, Stage 3:** event thresholding and merging into timed segments.
Bailey Malota	**Primary, Stage 2:** causal temporal model, conversion of clip-level labels to per-frame targets, trigger decoding, class-imbalance handling, calibration. **Secondary, Stage 1:** encoder comparison and embedding extraction.
Martin Balla	**Primary, Stage 3:** template construction, LLM selection, LoRA fine-tuning, robustness to incorrect upstream predictions. **Secondary, evaluation:** text-quality metrics and packaging the LLM for offline inference.

Schedule

Dates	Milestones
Sep 22	Proposal due. *Done.*
Sep 22 – Oct 19	Pipeline built and tested on public audio (pseudo-challenge set, synthetic scenes); encoder, decoder, and generator comparisons; challenge registration (opened Oct 7). *Offline container smoke test not yet done; moved to Oct 20–27.*
Oct 20	Data, rules, scoring tools, and baseline released (moved from Oct 14). **Checkpoint 1.**
Oct 20 – Nov 10	Adapt to the released rules; baseline reproduction; embedding extraction on ARC; dummy container; train decision model on real data; template system. Development leaderboard opens **Nov 4** (moved from Oct 21). **Checkpoint 2 (Nov 10):** end-to-end system with templates.
Nov 11 – 20	Finalize LLM and pretrained-model choices, checked against measured container size and latency; **team and resource declarations frozen Nov 20.**
Nov 21 – Dec 4	Integrate LLM generator into the container; ablations (templates vs. LLM, encoder choice); threshold tuning; stretch: encoder fine-tuning. **Final container frozen Dec 3** and validated on the leaderboard before it closes Dec 4.
Dec 5 – 11	No system changes; the leaderboard-validated container is submitted unchanged. Final report. Container and report **both due Dec 11.**
Dec 18	Final rankings and paper invitations announced.